

Week 6:

Python Fundamentals

– Part I

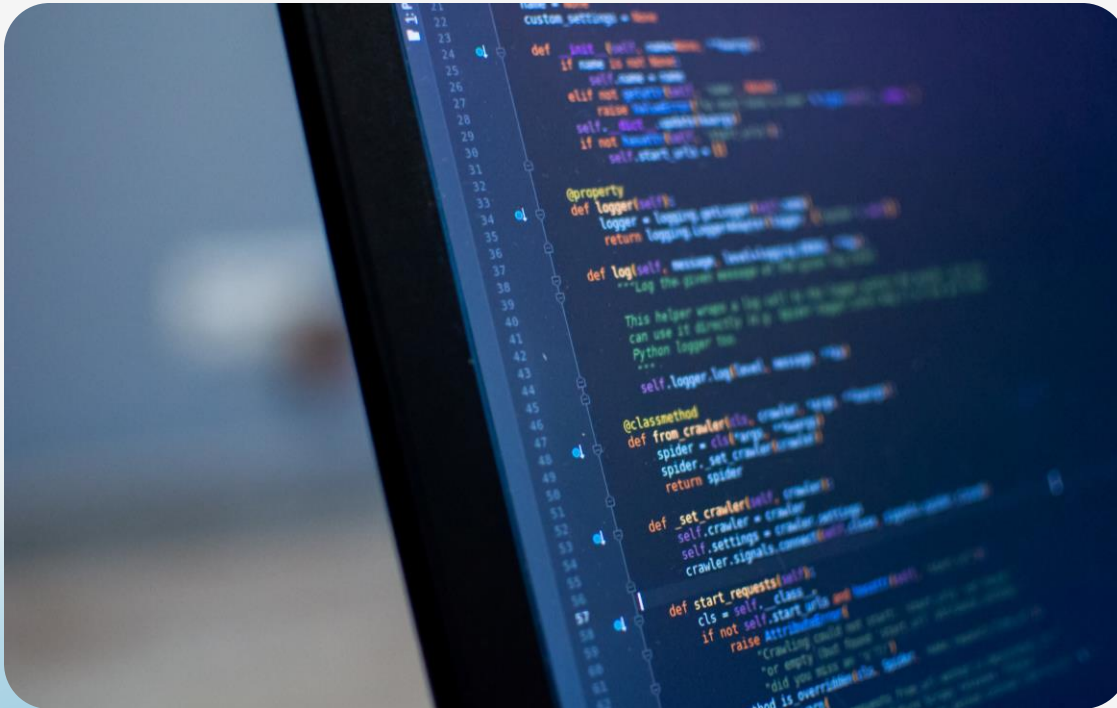


Topics Covered

- Python Fundamentals
- Mini Projects
- Case Study

Python Fundamentals

Control Structures in Python



Key Functions of Control Structures:

- Decision-Making: Implement conditional logic within programs
- Flow Control: Direct the order of statement execution
- Application Responsiveness: Build dynamic and interactive applications

Understanding if-else Statements

```
age = 18
if age >= 18:
    print("You are an adult.")
else:
    print("You are not an adult.")
```

if-else statements execute different code blocks based on whether a condition is true or false.

How if-else works:

- **Conditional Check:** Evaluates a specified condition.
- **True Execution:** Runs code within the if block if the condition is true.
- **False Execution:** Runs code within the else block if the condition is false.

Using if-elif-else for Multiple Conditions

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: D")
```

Key Features of if-elif-else:

- **Sequential Evaluation:** Conditions are checked in order
- **Exclusive Execution:** Only one code block executes (the first true condition)
- **Default Behaviour:** The else block executes if no other condition is met

Nested if Statements

```
num = 10
if num > 0:
    print("Positive number")
    if num % 2 == 0:
        print("Even number")
    else:
        print("Odd number")
else:
    print("Negative number")
```

Nested if statements embed if statements within other if or else blocks, creating hierarchical decision structures.

Purpose of Nested if Statements:

- **Implement Complex Logic:** Handle scenarios with multiple dependent conditions
- **Create Hierarchical Decisions:** Structure decision-making into levels of increasing specificity
- **Refine Conditional Checks:** Perform further checks based on the outcome of previous conditions

Practical Applications of Control Structures



Real-World Uses:

- **Form Validation:** Ensure accurate user input
- **Game Development:** Implement game logic and respond to player actions
- **Data Processing:** Perform different operations based on data characteristics
- **Automation Scripts:** Automate tasks by executing specific actions based on predefined conditions

Best Practices for Using Control Structures

Writing Effective Control Structures



- Keep conditions concise and easy to understand
- Minimise nesting to enhance readability
- Employ functions or dictionary-based "switches" for improved clarity and organisation in complex scenarios
- Use descriptive variable names to make conditions self-explanatory

Understanding File Handling in Python

Opening a File

File Access in Python

- Python's file handling enables interaction with computer files.
- It facilitates data storage and manipulation.
- The `open()` function is used for file handling.
- `open()` requires:
 - The filename.
 - The opening mode

Common File Opening Modes

- **'r' (Read):** Opens the file for reading (default mode)
- **'w' (Write):** Opens the file for writing, creating a new file or overwriting an existing one
- **'a' (Append):** Opens the file for appending, adding new data to the end
- **'b' (Binary):** Opens the file in binary mode

Understanding File Handling in Python

Reading Files

Code Example

```
file = open('example.txt', 'r')
content = file.read() # Read entire file
print(content)
file.close()
```

Key Reading Methods

- **read():** Reads the entire file content as a single string
- **readline():** Reads a single line from the file, including the newline character (`\n`)
- **readlines():** Reads all lines from the file and returns them as a list of strings

Understanding File Handling in Python

Writing and Appending to Files

Key Writing Methods

```
file = open('example.txt', 'w')  
file.write('Hello, world!\n')  
file.write('This is a new line.')  
file.close()
```

- **write():** Writes a single string to the file
- **writelines():** Writes a list of strings to the file

The Append Process

```
file = open('example.txt', 'a')  
file.write('\nAppending new content.')  
file.close()
```

- **Open in Append Mode:** Use the 'a' mode with the open() function
- The new data to the file will be added after the existing content

Using the with Statement for File Handling

Code Example

```
with open('example.txt', 'r') as file:  
    content = file.read()  
    print(content)  
# No need to call file.close()
```

Benefits of Using with

- **Automatic File Closure:** Guarantees the file is closed, even if errors occur
- **Simplified Syntax:** Reduces boilerplate code, making file handling more concise
- **Improved Code Reliability:** Minimises the risk of file corruption or resource leaks.

What are Comprehensions in Python?

Key Concepts

- Comprehensions offer a concise and efficient way to create lists, dictionaries, and sets in Python
- It improves readability and performance compared to traditional loops

Benefits of Comprehensions

- **Simplified Creation:** Streamline code for generating lists, dictionaries, and sets
- **Enhanced Readability:** Improve code clarity and expressiveness
- **Performance Efficiency:** Often provide faster execution than equivalent loops

List Comprehension

Code Example

```
# Traditional for loop approach
numbers = [1, 2, 3, 4, 5]
squares = []
for num in numbers:
    squares.append(num ** 2)
print(squares) # Output: [1, 4, 9, 16, 25]

# List comprehension approach
squares = [num ** 2 for num in numbers]
print(squares) # Output: [1, 4, 9, 16, 25]
```

Features of List Comprehensions

- It provide a concise way to create new lists from existing iterables
- **Syntax:**
[expression for item in iterable]
- **Optional Condition:**
[expression for item in iterable if condition]
- **Improved Readability:** More readable than equivalent for loops

Advanced List Comprehensions

Code Example

```
# List comprehension with a condition
even_squares = [num ** 2 for num in
range(10) if num % 2 == 0]
print(even_squares) # Output: [0, 4, 16, 36,
64]

# Nested list comprehension
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in
row]
print(flat) # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Advanced Features

- List comprehensions can be enhanced with conditional filtering and nesting for more complex list creation
- **Conditional Filtering:** [expression for item in iterable if condition] filters elements based on a condition
- **Nested Comprehensions:** Create multi-dimensional data structures or flatten nested lists

Dictionary Comprehension

Code Example

```
# Traditional approach
numbers = [1, 2, 3, 4]
squared_dict = {}
for num in numbers:
    squared_dict[num] = num ** 2
print(squared_dict) # Output: {1: 1, 2: 4, 3: 9, 4: 16}

# Dictionary comprehension approach
squared_dict = {num: num ** 2 for num
in numbers}
print(squared_dict) # Output: {1: 1, 2: 4, 3: 9, 4: 16}
```

Dictionary Comprehensions Features

- It provides a concise way to create dictionaries from iterables by transforming keys and values
- **Basic Syntax:** {key_expression: value_expression for item in iterable}
- **Optional Condition:** {key_expression: value_expression for item in iterable if condition}

Advanced Dictionary Comprehensions

Code Example

```
# Dictionary comprehension with a condition
squared_even_dict = {num: num ** 2 for
num in range(10) if num % 2 == 0}
print(squared_even_dict) # Output: {0: 0,
2: 4, 4: 16, 6: 36, 8: 64}
```

Advanced Features

- Dictionary comprehensions can include conditions to filter key-value pairs
- **Syntax with a condition:**
{key_expression: value_expression for item in iterable if condition}

Benefits of Using Comprehensions

Comprehensions offer several advantages over traditional loops when creating collections like lists, dictionaries, and sets.



- **Conciseness:** Achieve more with less code
- **Readability:** Enhance code clarity and understanding at a glance
- **Performance:** Often execute faster than equivalent loops and method calls
- **Expressiveness:** Combine filtering, mapping, and reducing operations in a single, readable line

Best Practices for Using Comprehensions

Writing Effective Comprehensions



- **Prioritise Readability:** Keep comprehensions simple and easy to understand
- **Avoid Over-Nesting:** If nesting significantly reduces readability, use standard loops instead
- **Use for Simple Operations:** Employ comprehensions for straightforward filtering and mapping tasks
- **Comment Complex Logic:** For more complex comprehensions, add comments to explain the logic

Object-Oriented Programming (OOP)

Key Principles

- Object-Oriented Programming (OOP) is a way of structuring code
- It organises code into reusable units called "objects"
- Each object combines data (its properties) and the actions it can perform (its methods)

Features and Benefits

- **Organizes Code:** Structures code into objects that represent real-world entities.
- **Promotes Reusability:** Encourages code reuse and modularity.
- **Supports Scalability:** Facilitates easier maintenance and expansion of code.
- **Four Core Principles:** Encapsulation, Abstraction, Inheritance, and Polymorphism

Understanding Classes and Objects

Code Example

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f'{self.name} says woof!'

my_dog = Dog('Buddy', 'Golden Retriever')
print(my_dog.bark()) # Output: Buddy says woof!
```

Key Concepts

- **Class:** A template for creating objects (like a blueprint)
- **Object:** An instance of a class (a specific creation based on the blueprint)
- **Attributes:** Variables that store data specific to an object
- **Methods:** Functions that define the object's behaviours or actions

The `__init__` Method

Constructor

Code Example

```
class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def display_info(self):
        return f"Car: {self.make} {self.model}"

car1 = Car('Toyota', 'Camry')
print(car1.display_info()) # Output: Car:
Toyota Camry
```

Key Functions of `__init__`

- **Automatic Invocation:** Automatically called when a new object is created (instantiated)
- **Attribute Initialisation:** Assigns initial values to an object's attributes
- **Object Setup:** Sets up the initial state of an object

Inheritance in Python

Code Example

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return f"{self.name} says woof!"

my_pet = Dog('Max')
print(my_pet.speak()) # Output: Max
                        says woof!
```

Key Concepts

- **Base Class (Parent):** The class whose properties are inherited.
- **Derived Class (Child):** The class that inherits properties from the base class.
- **Extending/Modifying Behaviour:** Allows for extending or modifying the behaviour of inherited classes.

Polymorphism in Python

Code Example

```
class Bird:
    def speak(self):
        return "Tweet!"
class Cat:
    def speak(self):
        return "Meow!"
animals = [Bird(), Cat()]
for animal in animals:
    print(animal.speak())

# Output:
# Tweet!
# Meow!
```

Key Aspects of Polymorphism

- **Unified Interface:** A single method name can be used across different classes
- **Varying Behaviour:** The method's implementation depends on the object's class
- **Achieved Through Overriding:** In Python, polymorphism is primarily achieved through method overriding.

Encapsulation and Abstraction

Code Example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        return self.__balance

account = BankAccount(1000)
print(account.deposit(500)) # Output: 1500
# print(account.__balance) # Error:
AttributeError
```

Key Aspects of Polymorphism

- **Encapsulation:** Bundles data (attributes) and methods that operate on that data within a single unit (a class), restricting direct access to some components
- **Abstraction:** Hides complex implementation details and exposes only essential information or interfaces to the user

Real-World Applications of OOP

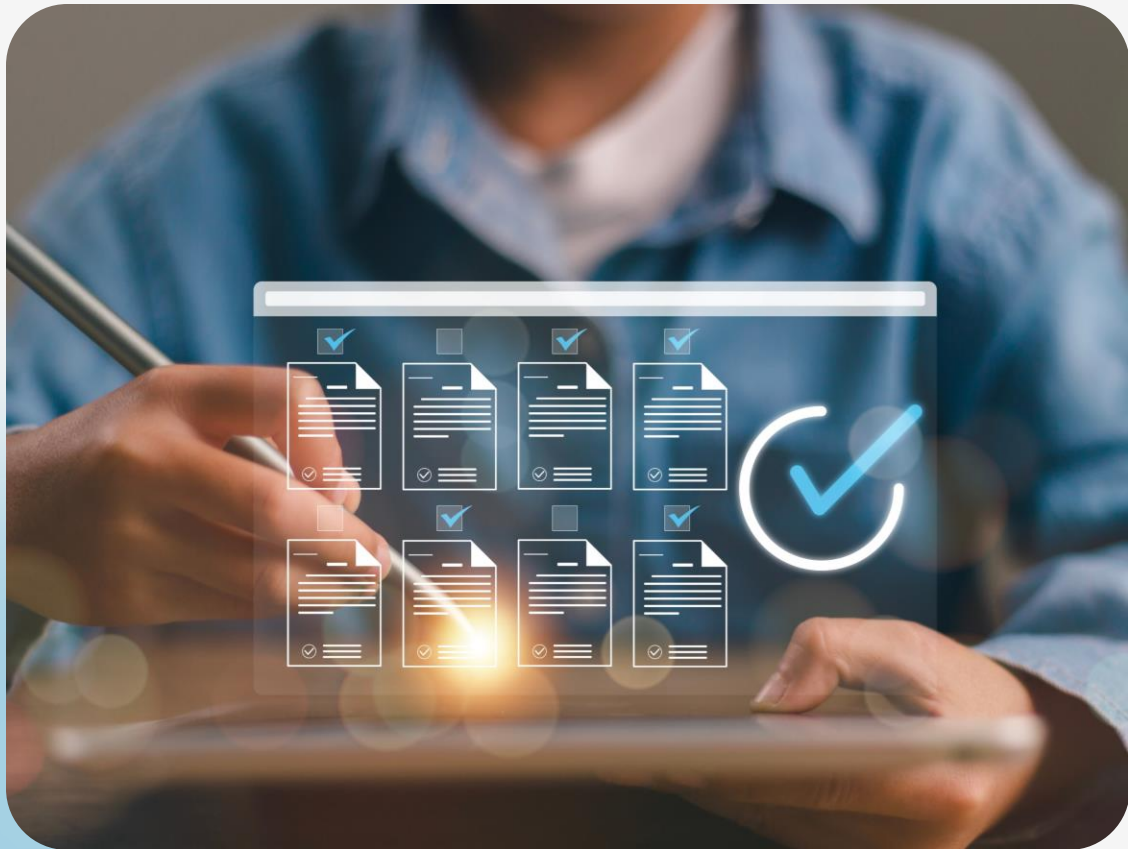
OOP principles ensure cleaner, maintainable, and robust code for diverse applications.



- **Game Development:** Creating game entities with defined behaviours and interactions
- **Web Development:** Structuring web applications using models for users, data, and server-side logic
- **Data Science:** Organising data analysis and manipulation workflows using classes and objects
- **GUI Applications:** Building interactive user interfaces with reusable UI components

Best Practices for OOP in Python

Adhering to these best practices will lead to cleaner, more maintainable, and robust object-oriented code.



- Use meaningful class and method names for better readability
- Single Responsibility Principle (SRP): Each class should have a single, well-defined purpose
- Avoid deep inheritance hierarchies; prefer composition over inheritance whenever possible
- Use docstrings to document classes and methods for better understanding

Generators in Python

Key Concepts

- Generators are a special type of iterable
- It produce a sequence of values one at a time, without storing the entire sequence in memory

Characteristics of Generators

- **yield Keyword:** Use yield instead of return to produce values iteratively
- **Lazy Evaluation:** Values are generated only when requested (on demand)
- **Memory Efficiency:** Ideal for working with large datasets, as they use minimal memory

Creating a Generator

Generators are created using functions, but instead of return, they use the yield keyword to produce a sequence of values iteratively.

Code Example

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
for number in countdown(5):  
    print(number)  
# Output: 5, 4, 3, 2, 1
```

Defining a Generator Function

- The *yield* keyword pauses execution and returns a value
- Each call to `next()` resumes execution from where it left off, yielding the next value
- Generators can be used directly in for loops.

Differences Between Generators and Regular Functions

Regular Functions:

- Use return to send a single value and then terminate.
- Start execution from the beginning each time they are called.
- Store all calculated values in memory (if creating a collection like a list).

Generators:

- Use yield to produce a value and pause execution, retaining their internal state
- Resume execution from where they left off when the next value is requested.
- Generate values on demand (lazy evaluation), saving memory.

Generator Expressions

Generator expressions offer a concise syntax for creating generators, similar to list comprehensions but using parentheses instead of square brackets.

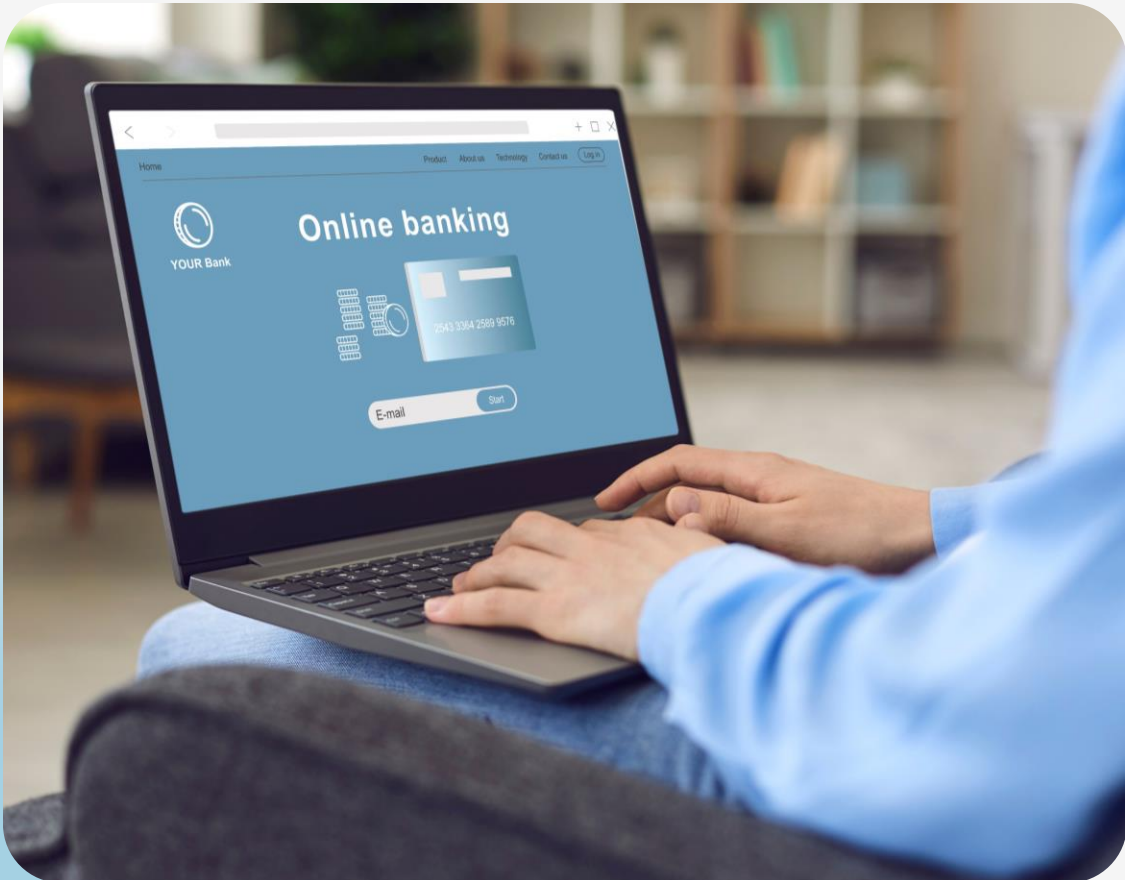
Code Example

```
squares = (x * x for x in range(5))  
for square in squares:  
    print(square)  
# Output: 0, 1, 4, 9, 16
```

Key Features

- **Concise Syntax:** (expression for item in iterable)
- **Memory Efficiency:** Generates values on demand, similar to generator functions.
- **Lazy Evaluation:** Values are produced only when iterated over.

Practical Applications of Generators



Real-World Uses:

Large Datasets: Process large files line by line without loading the entire file into memory

Infinite Sequences: Generate infinite sequences (e.g., Fibonacci numbers) without memory exhaustion

Data Pipelines: Create efficient data processing pipelines that generate intermediate results step by step

Asynchronous Programming: Use with `async` and `await` for efficient management of asynchronous tasks

Best Practices for Using Generators



Effective Generator Usages:

For Large Datasets: Use generators to process large datasets and avoid memory overload

itertools: Combine generators with the itertools module for complex iteration patterns

Generator Exhaustion: Be aware that once a generator is exhausted, it cannot be reused

Avoid Overuse: For simple, non-memory-intensive tasks, standard loops might be more appropriate

Why Python for Data Science?

Key Features

Python has become the leading language for data science due to its:

- Simplicity
- Readability
- Powerful libraries

Key Advantages

- **Extensive Community Support:** A large and active community provides ample resources and support
- **Powerful Libraries:** Specialized libraries (e.g., NumPy, Pandas, Scikit-learn) simplify complex data tasks
- **Seamless Integration:** Integrates well with other tools, languages, and technologies

Introduction to NumPy

Code Example

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr * 2)
```

Output: [2 4 6 8 10]

Key Features of NumPy

- N-dimensional Arrays (ndarray) for efficient storage and manipulation of numerical data in array format
- Offers a rich set of mathematical functions, including linear algebra, random number generation, and more
- Integrates seamlessly with C/C++ and Fortran code for optimized performance

Key Features of NumPy

Code Example

```
matrix = np.array([[1, 2], [3, 4]])  
print(np.linalg.inv(matrix)) # Inverse of  
the matrix  
print(np.mean(matrix)) # Mean of all  
elements
```

Core Functionalities

- **Array Manipulation:** Element-wise operations, broadcasting, slicing, and indexing
- **Math Functions:** Trigonometric, statistical, and linear algebra operations
- **Random Numbers:** Generation of random numbers for simulations and testing

Introduction to Pandas

Pandas is a Python library for efficient data manipulation and analysis with structured data.

Code Example

```
import pandas as pd
data = {'Name': ['John', 'Anna', 'Peter'],
        'Age': [28, 24, 35]}
df = pd.DataFrame(data)
print(df)
```

```
# Output:
#   Name  Age
# 0  John   28
# 1  Anna   24
# 2  Peter   35
```

Key Features of Pandas

- **DataFrame:** A 2D labeled data structure, similar to a table in SQL or Excel
- **Series:** A 1D labeled array, analogous to a column in a table
- **Data Handling:** Supports data alignment, missing data handling, and group operations

Key Features of Pandas

Code Example

```
# Filtering data
filtered_df = df[df['Age'] > 25]

# Grouping data
grouped = df.groupby('Age').mean()
```

Core Features

- **Data Manipulation:** Filtering, sorting, and aggregating data
- **Missing Data Handling:** Detecting, filling, and dropping missing values
- **GroupBy Operations:** Splitting data into groups and applying aggregate functions

Introduction to Matplotlib

Matplotlib is a Python library for static, animated, and interactive visualisations.

Code Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.title('Simple Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

Key Features

- **Diverse Plotting Options:** Supports a wide range of plot types
- **Highly Customisable:** Offers extensive control over plot appearance
- **Complex Layouts:** Enables the creation of subplots and complex figure layouts

Key Features of Matplotlib

Code Example

```
# Creating subplots
fig, axs = plt.subplots(2, 2)
axs[0, 0].plot(x, y)
axs[0, 1].scatter(x, y)
axs[1, 0].bar(x, y)
axs[1, 1].hist(y)
plt.show()
```

Core Capabilities

- **Diverse Plot Types:** Supports various plot types
- **Extensive Customisation:** Fine-grained control over plot appearance
- **Subplots and Layouts:** Enables the creation of complex figures with multiple subplots and custom layouts

Mini Projects

Mini Project 1: Library Management System

Project Overview



- Create a simple library management system
- Allow users to add, remove, search for, and borrow books
- Maintain a record of borrowed books and their borrowers

Action Items for Group Members: Member 1

Data Structures and Basic Functionality

Feature	Action
Lists and Tuples	Use a list of tuples to store book information (title, author, ISBN)
Basic Functions	Write functions for adding, removing, and listing books
Sets and Dictionaries	• Use a set to prevent duplicate books • Use a dictionary to map borrowed books to borrowers
Collaboration	Collaborate with Member 2 on data structure and duplicate handling

Action Items for Group Members: Member 2

Advanced Functionalities and File Handling

Feature	Action
File Handling	• Read and write to a file in CSV format for persistent storage • Load data from the file on startup and save changes on exit
Switch Cases	Use a dictionary-based approach for menu selection handling
Control Structures and Loops	• Create a menu-driven interface using loops and if/else statements • Allow users to add, remove, and borrow books
Collaboration	Collaborate with Member 1 on integrating file handling

Action Items for Group Members: Member 3

OOP and Generators

Feature	Action
OOP Concepts	• Create a Book class with attributes like title, author, ISBN, and status • Create a Library class with methods for managing books
List and Dictionary Comprehensions	• Filter books based on criteria (e.g., author, genre) • Create a summary report of borrowed books
Generators	Implement a generator to iterate over available books
Collaboration	Collaborate with Members 1 and 2 for integration

Mini Project 2: Personal Expense Tracker

Project Overview



- Develop a personal expense tracker
- Allow users to log expenses, categorize them, view summaries, and generate reports

Action Items for Group Members: Member 1

Data Structures and Basic Functionality

Feature	Action
Lists and Tuples	Use a list of tuples to store expense details (date, amount, category, description)
Basic Functions	Write functions to add, delete, and list expenses
Sets and Dictionaries	• Use a set for unique categories • Use a dictionary to map categories to expenses
Collaboration	Collaborate with Member 2 on data structures and functions

Action Items for Group Members: Member 2

User Interface and File Management

Feature

Action

File Handling

- Save and load expenses using CSV or JSON format
- Ensure data persistence between sessions

Switch Cases

- Use a dictionary-based approach for menu handling

Control Structures
and Loops

Create a command-line interface for navigating through options

Collaboration

Collaborate with Member 1 to integrate file handling

Action Items for Group Members: Member 3

Advanced Features and Reporting

Feature	Action
OOP Concepts	Create Expense and ExpenseTracker classes
List and Dictionary Comprehensions	• Filter expenses based on criteria • Generate summaries (e.g., total expenses per category)
Generators	Implement a generator for efficient report generation
Collaboration	Collaborate with Members 1 and 2 for seamless integration

Mini Project 3: Simple Quiz Application

Project Overview



- Create a quiz application where users take quizzes on various topics
- Record scores and display performance

Action Items for Group Members: Member 1

Data Management and Quiz Setup

Feature

Action

Lists and Tuples

Store quiz questions with options and correct answers in tuples

Basic Functions

Add, delete, and retrieve questions by topic

Sets and Dictionaries

- Use a set for quiz topics/categories
- Use a dictionary to map topics to their questions

Collaboration

Collaborate with Member 2 on quiz management

Action Items for Group Members: Member 2

User Interface and Scoring System

Feature	Action
File Handling	• Save and load user scores to/from a file • Ensure data persistence between sessions
Switch Cases	Use a dictionary-based approach for handling quiz options
Control Structures and Loops	Create a command-line interface for taking quizzes
Collaboration	Collaborate with Member 1 to integrate file handling

Action Items for Group Members: Member 3

Advanced Quiz Features and Reporting

Feature	Action
OOP Concepts	Create Question and Quiz classes
List and Dictionary Comprehensions	Filter questions and generate performance reports
Generators	Implement a generator for efficient question iteration
Collaboration	Collaborate with Members 1 and 2 for seamless integration

Case Study

Introduction to Case Study

Objective of the Report



Illustrate a real-world example where Python was utilized to solve a problem in an industry

Demonstrate how Python optimised a process or innovated a product/service

Provide a comprehensive overview including context, challenges faced, Python tools and techniques used, and results achieved

Case Study Report Outline

Introduction

- Industry & company overview
- Context: problem/need for python
- Significance of the case study

Background and Problem Statement

- Problem/challenge faced
- Criticality of addressing the problem
- Previous solutions & limitations

Role of Python

- Python as the solution tool
- Libraries/frameworks used
- Python-based solution
- Key features

Q&A Session



Open floor for questions and clarifications